Citraland Sport Finder

Panduan belajar: cara kerja aplikasi GIS pencari tempat olahraga terdekat di CitraLand Surabaya

Mata kuliah	Computer Graphics, Informatics, Universitas Ciputra
Teknologi	Python 3.12, Flask, GeoPandas, Shapely, OSMnx, Folium, Google Places API
Isi panduan	Gambaran umum, konsep GIS, alur data, penjelasan tiap file, cara menjalankan, soal latihan

1. Gambaran umum

Aplikasi ini menjawab satu pertanyaan: **tempat olahraga mana di CitraLand yang paling dekat dari alamatku?** Pengguna memilih jenis olahraga dan mengetik alamat awal. Aplikasi lalu menggambar rute jalan ke tempat terdekat di peta dan menampilkan jaraknya dalam kilometer.

Ada dua **layer** data spasial:

- **Layer 1 (titik):** tempat olahraga. Diambil dari Google Places API dan disimpan di `data/venues.geojson`.
- **Layer 2 (poligon):** batas kawasan CitraLand di `data/citraland_area.geojson`.

Lima langkah GIS dijalankan berurutan setiap kali pengguna menekan tombol:

Langkah	Fungsi	Konsep GIS
1	spatial_join	Spatial join: simpan titik yang berada di dalam poligon CitraLand
2	filter_by_sport	Filter atribut: pilih satu jenis olahraga
3	filter_open_now	Filter atribut dari jam buka (dihapus dari versi akhir)
4	rank_by_distance	Jarak garis lurus dalam meter, urut dari yang terdekat
5	route_to_nearest	Routing di jaringan jalan: jarak lewat jalan, bukan garis lurus

Catatan: README menyebut langkah 3, tetapi commit terakhir menghapus fungsi `filter_open_now` dan estimasi waktu tempuh. Kode saat ini hanya menjalankan langkah 1, 2, 4, dan 5.

2. Konsep GIS yang dipakai

2.1 Sistem koordinat (CRS)

Koordinat GPS dan Google memakai derajat (lintang dan bujur). Satu derajat bukan jarak yang tetap, jadi menghitung jarak langsung dari derajat menghasilkan angka yang salah. Karena itu aplikasi memakai dua CRS:

CRS	Kode	Dipakai untuk
WGS 84	EPSG:4326	Menyimpan dan menampilkan data (GeoJSON, peta Folium, input dari Google)
WGS 84 / UTM zona 49S	EPSG:32749	Semua perhitungan jarak, karena satuannya meter dan zona 49S mencakup Surabaya

Aturan di kode: data dibaca lalu langsung diubah ke UTM (`.to_crs(config.CRS_UTM)`) dan baru dikembalikan ke 4326 saat digambar di peta. Fungsi `rank_by_distance` melempar `ValueError` jika layer yang masuk bukan UTM, dan ada tes yang memeriksanya.

2.2 Spatial join

Google Text Search bisa mengembalikan tempat di luar CitraLand. Spatial join membuang titik yang tidak berada di dalam poligon:

```
joined = gpd.sjoin(venues, boundary[["geometry"]], how="inner", predicate="intersects")
```

`predicate="intersects"` berarti titik di tepi poligon juga dihitung masuk. `how="inner"` menyimpan hanya baris yang cocok. Setelah itu kolom `index_right` dibuang dan duplikat per (Tempat_ID, Jenis_Olahraga) dihapus, karena satu tempat bisa muncul untuk beberapa kata kunci.

2.3 Jarak garis lurus vs jarak lewat jalan

Tempat yang paling dekat secara garis lurus belum tentu paling dekat lewat jalan, misalnya kalau ada kanal atau jalan satu arah. Aplikasi memakai dua tahap:

- Urutkan semua tempat berdasarkan jarak garis lurus (murah dihitung).
- Hitung rute jalan hanya ke **3 kandidat teratas** (`ROUTE_CANDIDATES = 3`) lalu pilih yang rutenya terpendek.

Alasannya: menghitung rute ke semua tempat itu lambat, dan tempat terdekat lewat jalan hampir selalu ada di antara beberapa tempat terdekat secara garis lurus.

2.4 Routing dengan OSMnx

OSMnx mengunduh jaringan jalan (`network_type="drive"`) dari OpenStreetMap sebagai graf: persimpangan adalah *node*, ruas jalan adalah *edge* dengan atribut `length`. Rute terpendek dicari dengan `shortest_path(..., weight="length")`.

Titik awal dan tujuan jarang tepat di atas jalan, jadi keduanya di-*snap* ke node terdekat memakai `nearest_nodes`. Garis akhir digambar dari titik asli, lewat semua node, ke titik tujuan asli. Graf diambil dari area batas CitraLand ditambah buffer 2.500 m supaya pengguna dari luar kawasan (kos di Lontar atau Sambikerep) tetap bisa dirutekan. Graf disimpan di `data/graph.graphml`, jadi unduhan 1-2 menit hanya terjadi sekali.

3. Alur data dari klik sampai peta

```
Pengguna pilih olahraga + isi alamat, tekan "Tampilkan Rute"
        |
        v
app.py  index()          baca parameter: action, sport, address
        |
        v
gis.spatial_join         venues di dalam poligon CitraLand
        |
        v
gis.filter_by_sport      hanya olahraga yang dipilih
        |
        v   (jika action == "route")
resolve_origin           alamat -> (lat, lon) lewat Google, atau fallback ke Universitas Ciputra
        |
        v
gis.rank_by_distance     urut berdasarkan jarak garis lurus (meter, UTM)
        |
        v
gis.route_to_nearest     rute jalan ke 3 teratas, pilih yang terpendek
        |
        v
app.py  build_map        Folium: batas, titik venue, marker alamat, garis rute
        |
        v
templates/index.html     peta + panel hasil (nama, olahraga, jarak km)
```

Ada tiga mode halaman, dikendalikan parameter `action` di URL:

action	Kapan	Hasil
(kosong)	Pertama kali halaman dibuka	Semua tempat, tanpa filter dan tanpa rute
filter	Ganti dropdown olahraga	Titik difilter, tanpa menghitung rute
route	Tekan tombol Tampilkan Rute	Titik difilter, rute dan kartu hasil ditampilkan

4. Penjelasan tiap file

File	Peran
config.py	Konstanta bersama: path data, kode CRS, API key dari .env, koordinat fallback, pusat peta, dan kamus SPORTS.
gis.py	Inti GIS: load layer, spatial join, filter, ranking jarak, graf jalan, dan routing.
data_fetch.py	Mengambil tempat dari Places API, geocoding alamat, dan membangun poligon batas. Dijalankan dari command line.
app.py	Aplikasi Flask: satu route `/`, membangun peta Folium, mengirim data ke template.
templates/index.html	Halaman: peta di belakang, panel di kiri (bottom sheet di ponsel), form, kartu hasil.
static/style.css	Tampilan panel, tombol, kartu hasil, dan layout responsif.
tests/test_gis.py	14 tes pytest: spatial join, filter, urutan jarak, penggunaan UTM, filter nama olahraga, batas area.
data/*.geojson	venues (data Places asli), venues_sample (8 data contoh), citraland_area (batas dipakai), citraland_osm dan citraland_manual (cadangan).

4.1 config.py: kamus SPORTS

Satu kamus mengatur dropdown, warna marker, kata kunci pencarian, dan filter nama. Menambah olahraga baru cukup menambah satu entri:

```
"padel": {"label": "Padel", "query": "lapangan padel", "color": "#469990", "words": ["padel"]},
```

Kunci	Fungsi
label	Teks di dropdown dan legenda (Bahasa Indonesia)
query	Teks pencarian ke Places API, ditambah "CitraLand Surabaya"
color	Warna marker di peta
words	Kata di nama tempat yang menandakan olahraga ini
not_words	Kata yang menyingkirkan olahraga ini walau ada kata cocok (tenis vs tenis meja)

4.2 gis.py: fungsi per fungsi

Fungsi	Masukan	Keluaran
load_boundary	-	Poligon CitraLand dalam UTM. Urutan sumber: citraland_area, cache OSM, unduh dari OSM, poligon manual.
load_venues	-	Titik venue dalam UTM: cache Google Places jika ada, kalau tidak data contoh.
point_from_latlon	lat, lon	GeoSeries satu titik pengguna dalam UTM.
spatial_join	venues, boundary	Venue di dalam atau di tepi poligon, tanpa duplikat.

Fungsi	Masukan	Keluaran
filter_by_sport	venues, sport	Semua jika "semua", selain itu filter kolom Jenis_Olahraga.
rank_by_distance	venues, titik pengguna	Kolom Jarak_m (meter), diurutkan naik.
get_graph	boundary	Graf jalan UTM, di-cache di disk dan di memori (variabel global).
_route	graf, titik awal, tujuan	Dict berisi length_m dan line (LineString), atau None jika tidak ada rute.
route_to_nearest	ranked, titik pengguna, graf	Tabel dengan Jarak_Rute_m dan rute terbaik, garisnya diubah ke 4326 untuk peta.

4.3 data_fetch.py

- **search_places:** satu Text Search per olahraga, dengan *location bias* lingkaran radius 6 km di pusat CitraLand. Jam buka disimpan sebagai teks JSON karena properti GeoJSON harus datar.
- **names_other_sport:** Text Search longgar ("lapangan futsal" juga mengembalikan lapangan tenis). Fungsi ini membuang hasil yang namanya jelas milik olahraga lain, tetapi mempertahankan nama yang tidak menyebut olahraga apa pun (misal "Citraland Square Park").
- **geocode:** mencoba Geocoding API; jika ditolak (REQUEST_DENIED) beralih ke Places Text Search.
- **build_area:** menggabungkan dua poligon OSM (Citraland RW 07 Made dan Citraland Utara) dengan poligon gambar tangan untuk Northwest dan The GreenLake, lalu `buffer(50).buffer(-50)` menutup celah tipis dan lubang dibuang.

4.4 app.py

- **BOUNDARY dan VENUES_ALL** dimuat sekali saat server start, bukan tiap request.
- **resolve_origin:** tanpa API key atau saat geocoding gagal, aplikasi memakai Universitas Ciputra dan menambahkan peringatan ke panel.
- **build_map:** menggambar batas, CircleMarker per venue, marker rumah untuk alamat awal, dan rute (garis putih tebal di bawah garis gelap supaya terbaca di atas jalan). Zoom dilakukan di browser lewat `fitBounds` agar padding mengikuti ukuran layar.
- **km_text:** mengubah meter ke teks seperti "1,25 km" dengan koma desimal.

5. Cara menjalankan

```
py -3.12 -m venv .venv
.venv\Scripts\pip install -r requirements.txt
copy .env.example .env
.venv\Scripts\flask --app app run
```

Buka `http://127.0.0.1:5000`. Repo sudah menyertakan `data/venues.geojson` (data asli), jadi aplikasi langsung jalan tanpa API key. Tanpa key, alamat awal selalu Universitas Ciputra.

- **Mengisi API key (opsional):** isi `GOOGLE_MAPS_API_KEY` di file `.env` supaya alamat bisa di-geocode. Aktifkan Places API (New) dan Geocoding API di Google Cloud. File .env tidak ikut di repo.
- **Ambil ulang data venue:** `python data_fetch.py --refresh` (butuh key).
- **Bangun ulang batas area:** `python data_fetch.py --boundary`.
- **Rute pertama lambat:** graf jalan OSM diunduh sekitar 1-2 menit, lalu di-cache.
- **Jalankan tes:** `.venv\Scripts\python -m pytest`.

Batasan yang perlu diketahui

- Batas CitraLand untuk Northwest, The GreenLake, dan celah antar bagian adalah **perkiraan gambar tangan** (27 km2), karena OSM tidak punya poligonnya.
- Data Google Places bisa salah label. Contoh: Private Fitness berlabel "Ngagel" tetapi koordinatnya masuk poligon CitraLand Utara, jadi dihitung sebagai CitraLand.
- Jarak rute memakai panjang jalan, bukan waktu tempuh. Estimasi waktu sengaja dihapus karena tidak akurat.

6. Soal latihan

Jawab dulu tanpa melihat kunci, lalu cocokkan.

1. Mengapa jarak dihitung di EPSG:32749 dan bukan EPSG:4326?

Jawaban: EPSG:4326 memakai derajat, yang bukan satuan jarak tetap. EPSG:32749 (UTM 49S) memakai meter dan akurat untuk Surabaya.

2. Apa beda predicate "intersects" dan "within" pada spatial join?

Jawaban: "intersects" menyertakan titik di tepi poligon. "within" mengharuskan titik benar-benar di dalam, sehingga titik di tepi ikut terbuang.

3. Mengapa rute dihitung ke 3 kandidat, bukan 1 atau semua?

Jawaban: Terdekat secara garis lurus belum tentu terdekat lewat jalan, jadi satu kandidat bisa salah. Semua kandidat terlalu lambat. Tiga adalah kompromi.

4. Mengapa titik awal di-snap ke node jalan terdekat?

Jawaban: Graf hanya punya node di persimpangan. Titik pengguna atau venue jarang tepat di jalan, dan venue baru bisa berada di jalan masuk yang belum dipetakan OSM.

5. Apa fungsi names_other_sport?

Jawaban: Membuang hasil Places yang namanya jelas milik olahraga lain, tetapi mempertahankan nama yang tidak menyebut olahraga apa pun.

6. Apa yang terjadi jika API key kosong?

Jawaban: Data venue tetap dari cache, dan alamat awal memakai koordinat Universitas Ciputra dengan peringatan di panel.

7. Mengapa graf jalan diberi buffer 2.500 m?

Jawaban: Agar pengguna yang berada di luar CitraLand (Lontar, Sambikerep) tetap punya rute ke venue di dalam kawasan.

8. Mengapa garis rute digambar dua kali di build_map?

Jawaban: Garis putih tebal di bawah garis gelap berfungsi sebagai tepi, sehingga rute tetap terbaca di atas jalan dan marker.

9. Mengapa peta difit-bounds di browser, bukan di Python?

Jawaban: Padding bergantung pada lebar layar: panel di kiri pada desktop dan bottom sheet pada ponsel. Python tidak tahu ukuran layar.

10. Bagaimana menambah olahraga baru?

Jawaban: Tambah satu entri di config.SPORTS (label, query, color, words), lalu jalankan data_fetch.py --refresh. Dropdown dan legenda terisi otomatis.

7. Glosarium

Istilah	Arti
Layer	Satu lapisan data spasial, misalnya titik venue atau poligon batas.

Istilah	Arti
GeoJSON	Format file teks untuk data spasial: geometri ditambah atribut.
GeoDataFrame	Tabel pandas dengan kolom geometri, dari GeoPandas.
CRS	Coordinate Reference System: aturan yang mengubah angka koordinat menjadi posisi di bumi.
Spatial join	Menggabungkan atau menyaring dua layer berdasarkan hubungan ruang, bukan kolom yang sama.
Node dan edge	Persimpangan dan ruas jalan dalam graf jaringan jalan.
Snap	Memindahkan titik ke objek terdekat, di sini node jalan terdekat.
Geocoding	Mengubah alamat teks menjadi koordinat.
Location bias	Petunjuk ke Places API agar hasil pencarian diprioritaskan di sekitar lokasi tertentu.
Folium	Library Python yang membuat peta interaktif Leaflet sebagai HTML.